```
username = user
password = password
```

Other options can be specified on the command-line; their full list is available in the `mount.cifs(1)` manual page. Two options in particular can be interesting: `uid` and `gid` allow forcing the owner and group of files available on the mount, so as not to restrict access to root.

A mount of a Windows share can also be configured in `/etc/fstab`:

```
//server/shared /shared cifs credentials=/etc/smb-credentials
```

Unmounting a SMB/CIFS share is done with the standard `umount` command.

Printing on a Shared Printer

CUPS is an elegant solution for printing from a Linux workstation to a printer shared by a Windows machine. When the *smbclient* is installed, CUPS allows installing Windows shared printers automatically.

Here are the required steps:

- Enter the CUPS configuration interface: http://localhost:631/admin
- Click on "Add Printer".
- Choose the printer device, pick "Windows Printer via SAMBA".
- Enter the connection URI for the network printer. It should look like the following:

  smb://*user*:*password*@*server*/*printer*.
- Enter the name that will uniquely identify this printer. Then enter the description and location of the printer. Those are the strings that will be shown to end users to help them identify the printers.
- Indicate the manufacturer/model of the printer, or directly provide a working printer description file (PPD).

Voilà, the printer is operational!

11.6. HTTP/FTP Proxy

An HTTP/FTP proxy acts as an intermediary for HTTP and/or FTP connections. Its role is twofold:

- Caching: recently downloaded documents are copied locally, which avoids multiple downloads.

- Filtering server: if use of the proxy is mandated (and outgoing connections are blocked unless they go through the proxy), then the proxy can determine whether or not the request is to be granted.

Falcot Corp selected Squid as their proxy server.

11.6.1. Installing

The *squid*[2] Debian package only contains the modular (caching) proxy. Turning it into a filtering server requires installing the additional *squidguard* package. In addition, *squid-cgi* provides a querying and administration interface for a Squid proxy.

Prior to installing, care should be taken to check that the system can identify its own complete name: the `hostname -f` must return a fully-qualified name (including a domain). If it does not, then the `/etc/hosts` file should be edited to contain the full name of the system (for instance, arrakis.falcot.com). The official computer name should be validated with the network administrator in order to avoid potential name conflicts.

11.6.2. Configuring a Cache

Enabling the caching server feature is a simple matter of editing the `/etc/squid/squid.conf` configuration file and allowing machines from the local network to run queries through the proxy. The following example shows the modifications made by the Falcot Corp administrators:

Example 11.22 *The `/etc/squid/squid.conf` file (excerpts)*

```
# INSERT YOUR OWN RULE(S) HERE TO ALLOW ACCESS FROM YOUR CLIENTS
#
include /etc/squid/conf.d/*

# Example rule allowing access from your local networks.
# Adapt localnet in the ACL section to list your (internal) IP networks
# from where browsing should be allowed

acl our_networks src 192.168.1.0/24 192.168.2.0/24
http_access allow our_networks
http_access allow localhost
# And finally deny all other access to this proxy
http_access deny all
```

[2] The *squid3* package, providing Squid until Debian *Jessie*, is now a transitional package and will automatically install *squid*.

11.6.3. Configuring a Filter

`squid` itself does not perform the filtering; this action is delegated to `squidGuard`. The former must then be configured to interact with the latter. This involves adding the following directive to the `/etc/squid/squid.conf` file:

```
url_rewrite_program /usr/bin/squidGuard -c /etc/squid/squidGuard.conf
```

The `/usr/lib/cgi-bin/squidGuard.cgi` CGI program also needs to be installed, using `/usr/share/doc/squidguard/examples/squidGuard.cgi.gz` as a starting point. Required modifications to this script are the `$proxy` and `$proxymaster` variables (the name of the proxy and the administrator's contact email, respectively). The `$image` and `$redirect` variables should point to existing images representing the rejection of a query.

The filter is enabled with the `service squid reload` command. However, since the *squidguard* package does no filtering by default, it is the administrator's task to define the policy. This can be done by creating the `/etc/squid/squidGuard.conf` file (using `/etc/squidguard/squidGuard.conf.default` as template if required).

The working database must be regenerated with `update-squidguard` after each change of the `squidGuard` configuration file (or one of the lists of domains or URLs it mentions). The configuration file syntax is documented on the following website:

➡ http://www.squidguard.org/Doc/configure.html

ALTERNATIVE

E2guardian (a DansGuardian Fork)

The *e2guardian* package, a DansGuardian fork, is an alternative to *squidguard*. This software does not simply handle a blacklist of forbidden URLs, but it can take advantage of the PICS[3] (*Platform for Internet Content Selection*) to decide whether a page is acceptable by dynamic analysis of its contents.

11.7. LDAP Directory

OpenLDAP is an implementation of the LDAP protocol; in other words, it is a special-purpose database designed for storing directories. In the most common use case, using an LDAP server allows centralizing management of user accounts and the related permissions. Moreover, an LDAP database is easily replicated, which allows setting up multiple synchronized LDAP servers. When the network and the user base grows quickly, the load can then be balanced across several servers.

LDAP data is structured and hierarchical. The structure is defined by "schemas" which describe the kind of objects that the database can store, with a list of all their possible attributes. The syntax used to refer to a particular object in the database is based on this structure, which explains its complexity.

[3] PICS has been superseded by the *Protocol for Web Description Resources* (POWDER system: https://www.w3.org/2009/08/pics_superseded.html.

11.7.1. Installing

The *slapd* package contains the OpenLDAP server. The *ldap-utils* package includes command-line tools for interacting with LDAP servers.

Installing *slapd* usually asks only for the administrator's password and the resulting database is unlikely to suit your needs. Fortunately a simple `dpkg-reconfigure slapd` will let you reconfigure the LDAP database with more details:

- Omit OpenLDAP server configuration? No, of course, we want to configure this service.
- DNS domain name: "falcot.com".
- Organization name: "Falcot Corp".
- An administrative passwords needs to be typed in.
- Database backend to use: "MDB".
- Do you want the database to be removed when *slapd* is purged? No. No point in risking losing the database in case of a mistake.
- Move old database? This question is only asked when the configuration is attempted while a database already exists. Only answer "yes" if you actually want to start again from a clean database, for instance if you run `dpkg-reconfigure slapd` right after the initial installation.

BACK TO BASICS

LDIF format

An LDIF file (*LDAP Data Interchange Format*) is a portable text file describing the contents of an LDAP database (or a portion thereof); this can then be used to inject the data into any other LDAP server.

A minimal database is now configured, as demonstrated by the following query:

```
$ ldapsearch -x -b dc=falcot,dc=com
# extended LDIF
#
# LDAPv3
# base <dc=falcot,dc=com> with scope subtree
# filter: (objectclass=*)
# requesting: ALL
#

# falcot.com
dn: dc=falcot,dc=com
objectClass: top
objectClass: dcObject
objectClass: organization
o: Falcot Corp
dc: falcot

# admin, falcot.com
```

```
dn: cn=admin,dc=falcot,dc=com
objectClass: simpleSecurityObject
objectClass: organizationalRole
cn: admin
description: LDAP administrator

# search result
search: 2
result: 0 Success

# numResponses: 3
# numEntries: 2
```

The query returned two objects: the organization itself, and the administrative user.

11.7.2. Filling in the Directory

Since an empty database is not particularly useful, we are going to inject into it all the existing directories; this includes the users, groups, services and hosts databases.

The *migrationtools* package provides a set of scripts dedicated to extract data from the standard Unix directories (`/etc/passwd`, `/etc/group`, `/etc/services`, `/etc/hosts` and so on), convert this data, and inject it into the LDAP database.

Once the package is installed, the `/etc/migrationtools/migrate_common.ph` must be edited; the `IGNORE_UID_BELOW` and `IGNORE_GID_BELOW` options need to be enabled (uncommenting them is enough), and `DEFAULT_MAIL_DOMAIN`/`DEFAULT_BASE` need to be updated.

The actual migration operation is handled by the `migrate_all_online.sh` command, as follows:

```
# cd /usr/share/migrationtools
# LDAPADD="/usr/bin/ldapadd -c" ETC_ALIASES=/dev/null ./migrate_all_online.sh
```

The `migrate_all_online.sh` asks a few questions about the LDAP database into which the data is to be migrated. Table 11.1 summarizes the answers given in the Falcot use-case.

Question	Answer
X.500 naming context	dc=falcot,dc=com
LDAP server hostname	localhost
Manager DN	cn=admin,dc=falcot,dc=com
Bind credentials	the administrative password
Create DUAConfigProfile	no

Table 11.1 *Answers to questions asked by the `migrate_all_online.sh` script*

We deliberately ignore migration of the `/etc/aliases` file, since the standard schema as provided by Debian does not include the structures that this script uses to describe email aliases. Should we want to integrate this data into the directory, the `/etc/ldap/schema/misc.schema` file should be added to the standard schema.

TOOL

Browsing an LDAP directory

The `jxplorer` command (in the package of the same name) is a graphical tool allowing to browse and edit an LDAP database. It is an interesting tool that provides an administrator with a good overview of the hierarchical structure of the LDAP data.

Also note the use of the -c option to the `ldapadd` command; this option requests that processing doesn't stop in case of error. Using this option is required because converting the `/etc/services` often generates a few errors that can safely be ignored.

11.7.3. Managing Accounts with LDAP

Now the LDAP database contains some useful information, the time has come to make use of this data. This section focuses on how to configure a Linux system so that the various system directories use the LDAP database.

Configuring NSS

The NSS system (Name Service Switch, see sidebar "NSS and system databases" page 174) is a modular system designed to define or fetch information for system directories. Using LDAP as a source of data for NSS requires installing the *libnss-ldap* package. Its installation asks a few questions; the answers are summarized in Table 11.2 .

Question	Answer
LDAP server Uniform Resource Identifier	ldapi://ldap.falcot.com
Distinguished name of the search base	dc=falcot,dc=com
LDAP version to use	3
LDAP account for root	cn=admin,dc=falcot,dc=com
LDAP root account password	the administrative password
Allow LDAP admin account behave like local root?	yes
Does the LDAP database require login?	no

Table 11.2 *Configuring the* libnss-ldap *package*

The `/etc/nsswitch.conf` file then needs to be modified, so as to configure NSS to use the freshly-installed `ldap` module. You can use the example provided in /usr/share/doc/libnss-ldap/examples/nsswitch.ldap or edit your existing configuration.

Example 11.23 *The /etc/nsswitch.conf file*

```
#ident $Id: nsswitch.ldap,v 2.4 2003/10/02 02:36:25 lukeh Exp $
#
# An example file that could be copied over to /etc/nsswitch.conf; it
# uses LDAP conjunction with files.
#
# "hosts:" and "services:" in this file are used only if the
# /etc/netconfig file has a "-" for nametoaddr_libs of "inet" transports.

# the following lines obviate the "+" entry in /etc/passwd and /etc/group.
passwd:                 files ldap
shadow:                 files ldap
group:                 files ldap

# consult DNS first, we will need it to resolve the LDAP host. (If we
# can't resolve it, we're in infinite recursion, because libldap calls
# gethostbyname(). Careful!)
hosts:                dns ldap

# LDAP is nominally authoritative for the following maps.
services:   ldap [NOTFOUND=return] files
networks:   ldap [NOTFOUND=return] files
protocols:  ldap [NOTFOUND=return] files
rpc:        ldap [NOTFOUND=return] files
ethers:     ldap [NOTFOUND=return] files

# no support for netmasks, bootparams, publickey yet.
netmasks:   files
bootparams: files
publickey:  files
automount:  files

# I'm pretty sure nsswitch.conf is consulted directly by sendmail,
# here, so we can't do much here. Instead, use bbense's LDAP
# rules ofr sendmail.
aliases:    files
sendmailvars:   files

# Note: there is no support for netgroups on Solaris (yet)
netgroup:   ldap [NOTFOUND=return] files
```

The `ldap` module is usually inserted before others, and it will therefore be queried first. The notable exception is the hosts service since contacting the LDAP server requires consulting DNS first (to resolve ldap.falcot.com). Without this exception, a hostname query would try to ask the

LDAP server; this would trigger a name resolution for the LDAP server, and so on in an infinite loop.

If the LDAP server should be considered authoritative (and the local files used by the `files` module disregarded), services can be configured with the following syntax:

service: ldap [NOTFOUND=return] files.

If the requested entry does not exist in the LDAP database, the query will return a "not existing" reply even if the resource does exist in one of the local files; these local files will only be used when the LDAP service is down.

Configuring PAM

This section describes a PAM configuration (see sidebar "`/etc/environment` and `/etc/default/locale`" page 161) that will allow applications to perform the required authentications against the LDAP database.

CAUTION

Broken authentication

Changing the standard PAM configuration used by various programs is a sensitive operation. A mistake can lead to broken authentication, which could prevent logging in. Keeping a root shell open is therefore a good precaution. If configuration errors occur, they can be then fixed and the services restarted with minimal effort.

The LDAP module for PAM is provided by the *libpam-ldap* package. Installing this package asks a few questions very similar to those in *libnss-ldap*; some configuration parameters (such as the URI for the LDAP server) are even actually shared with the *libnss-ldap* package. Answers are summarized in Table 11.3 .

Question	Answer
Allow LDAP admin account to behave like local root?	Yes. This allows using the usual `passwd` command for changing passwords stored in the LDAP database.
Does the LDAP database require logging in?	no
LDAP account for root	cn=admin,dc=falcot,dc=com
LDAP root account password	the LDAP database administrative password
Local encryption algorithm to use for passwords	crypt

Table 11.3 *Configuration of* libpam-ldap

Installing *libpam-ldap* automatically adapts the default PAM configuration defined in the `/etc/pam.d/common-auth`, `/etc/pam.d/common-password` and `/etc/pam.d/common-account` files. This mechanism uses the dedicated `pam-auth-update` tool (provided by the *libpam-runtime* package). This tool can also be run by the administrator should they wish to enable or disable PAM modules.

By default, the LDAP protocol transits on the network as cleartext; this includes the (encrypted) passwords. Since the encrypted passwords can be extracted from the network, they can be vulnerable to dictionary-type attacks. This can be avoided by using an extra encryption layer; enabling this layer is the topic of this section.

Configuring the Server The first step is to create a key pair (comprising a public key and a private key) for the LDAP server. The Falcot administrators reuse *easy-rsa* to generate it (see section 10.2.2, "Public Key Infrastructure: *easy-rsa*" page 243). Running `./easyrsa build-server-full ldap.falcot.com nopass` will ask you about the "common name". The answer to that question *must* be the fully-qualified hostname for the LDAP server; in our case, ldap.falcot.com.

This command creates a certificate in the `pki/issued/ldap.falcot.com.crt` file; the corresponding private key is stored in `pki/private/ldap.falcot.com.key`.

Now these keys have to be installed in their standard location, and we must make sure that the private file is readable by the LDAP server which runs under the openldap user identity:

```
# adduser openldap ssl-cert
Adding user ‘openldap’ to group ‘ssl-cert’ ...
Adding user openldap to group ssl-cert
Done.
# mv pki/private/ldap.falcot.com.key /etc/ssl/private/ldap.falcot.com.key
# chown root:ssl-cert /etc/ssl/private/ldap.falcot.com.key
# chmod 0640 /etc/ssl/private/ldap.falcot.com.key
# ./eassyrsa gen-dh

Note: using Easy-RSA configuration from: ./vars

Using SSL: openssl OpenSSL 1.1.1c  28 May 2019
Generating DH parameters, 2048 bit long safe prime, generator 2
This is going to take a long time
.....................................................+.....................................
    ➥
[...]
DH parameters of size 2048 created at /home/roland/pki/dh.pem

# mv pki/dh.pem /etc/ssl/certs/ldap.falcot.com.pem
```

The `slapd` daemon also needs to be told to use these keys for encryption. The LDAP server configuration is managed dynamically: the configuration can be updated with normal LDAP operations on the cn=config object hierarchy, and the server updates `/etc/ldap/slapd.d` in real time to make the configuration persistent. `ldapmodify` is thus the right tool to update the configuration:

Example 11.24 *Configuring `slapd` for encryption*

```
# cat >ssl.ldif <<END
dn: cn=config
changetype: modify
add: olcTLSCertificateFile
olcTLSCertificateFile: /etc/ssl/certs/ldap.falcot.com.pem
-
add: olcTLSCertificateKeyFile
olcTLSCertificateKeyFile: /etc/ssl/private/ldap.falcot.com.key
-
END
# ldapmodify -Y EXTERNAL -H ldapi:/// -f ssl.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
modifying entry "cn=config"
```

TOOL

`ldapvi` to edit an LDAP directory

With `ldapvi`, you can display an LDIF output of any part of the LDAP directory, make some changes in the text editor, and let the tool do the corresponding LDAP operations for you.

It is thus a convenient way to update the configuration of the LDAP server, simply by editing the `cn=config` hierarchy.

```
# ldapvi -Y EXTERNAL -h ldapi:/// -b cn=config
```

The last step for enabling encryption involves changing the `SLAPD_SERVICES` variable in the `/etc/default/slapd` file. We'll play it safe and disable unsecured LDAP altogether.

Example 11.25 *The `/etc/default/slapd` file*

```
# Default location of the slapd.conf file or slapd.d cn=config directory. If
# empty, use the compiled-in default (/etc/ldap/slapd.d with a fallback to
# /etc/ldap/slapd.conf).
SLAPD_CONF=

# System account to run the slapd server under. If empty the server
# will run as root.
SLAPD_USER="openldap"

# System group to run the slapd server under. If empty the server will
# run in the primary group of its user.
SLAPD_GROUP="openldap"
```

```
# Path to the pid file of the slapd server. If not set the init.d script
# will try to figure it out from $SLAPD_CONF (/etc/ldap/slapd.conf by
# default)
SLAPD_PIDFILE=

# slapd normally serves ldap only on all TCP-ports 389. slapd can also
# service requests on TCP-port 636 (ldaps) and requests via unix
# sockets.
# Example usage:
# SLAPD_SERVICES="ldap://127.0.0.1:389/ ldaps:/// ldapi:///"
SLAPD_SERVICES="ldaps:/// ldapi:///"

# If SLAPD_NO_START is set, the init script will not start or restart
# slapd (but stop will still work).  Uncomment this if you are
# starting slapd via some other means or if you don't want slapd normally
# started at boot.
#SLAPD_NO_START=1

# If SLAPD_SENTINEL_FILE is set to path to a file and that file exists,
# the init script will not start or restart slapd (but stop will still
# work).  Use this for temporarily disabling startup of slapd (when doing
# maintenance, for example, or through a configuration management system)
# when you don't want to edit a configuration file.
SLAPD_SENTINEL_FILE=/etc/ldap/noslapd

# For Kerberos authentication (via SASL), slapd by default uses the system
# keytab file (/etc/krb5.keytab).  To use a different keytab file,
# uncomment this line and change the path.
#export KRB5_KTNAME=/etc/krb5.keytab

# Additional options to pass to slapd
SLAPD_OPTIONS=""
```

Configuring the Client On the client side, the configuration for the *libpam-ldap* and *libnss-ldap* modules needs to be modified to use an ldaps:// URI.

LDAP clients also need to be able to authenticate the server. In a X.509 public key infrastructure, public certificates are signed by the key of a certificate authority (CA). With *easy-rsa*, the Falcot administrators have created their own CA and they now need to configure the system to trust the signatures of Falcot's CA. This can be done by putting the CA certificate in `/usr/local/share/ca-certificates` and running `update-ca-certificates`.

```
# cp pki/ca.crt /usr/local/share/ca-certificates/falcot.crt
# update-ca-certificates
Updating certificates in /etc/ssl/certs... 1 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d....
```

```
Adding debian:falcot.pem
done.
done.
```

Last but not least, the default LDAP URI and default base DN used by the various command line tools can be modified in `/etc/ldap/ldap.conf`. This will save quite some typing.

Example 11.26 *The `/etc/ldap/ldap.conf` file*

```
#
# LDAP Defaults
#

# See ldap.conf(5) for details
# This file should be world readable but not world writable.

BASE   dc=falcot,dc=com
URI    ldaps://ldap.falcot.com

#SIZELIMIT      12
#TIMELIMIT      15
#DEREF          never

# TLS certificates (needed for GnuTLS)
TLS_CACERT      /etc/ssl/certs/ca-certificates.crt
```

11.8. Real-Time Communication Services

Real-Time Communication (RTC) services include voice, video/webcam, instant messaging (IM) and desktop sharing. This chapter gives a brief introduction to three of the services required to operate RTC, including a TURN server, SIP server and XMPP server. Comprehensive details of how to plan, install and manage these services are available in the Real-Time Communications Quick Start Guide which includes examples specific to Debian.

➡ https://rtcquickstart.org

Both SIP and XMPP can provide the same functionality. SIP is slightly more well known for voice and video while XMPP is traditionally regarded as an IM protocol. In fact, they can both be used for any of these purposes. To maximize connectivity options, it is recommended to run both in parallel.

These services rely on X.509 certificates both for authentication and confidentiality purposes. See section 10.2, "X.509 certificates" page 240 for more information.

11.8.1. DNS settings for RTC services

RTC services require DNS SRV and NAPTR records. A sample configuration that can be placed in the zone file for falcot.com:

```
; the server where everything will run
server1            IN     A      198.51.100.19
server1            IN     AAAA   2001:DB8:1000:2000::19

; IPv4 only for TURN for now, some clients are buggy with IPv6
turn-server        IN     A      198.51.100.19

; IPv4 and IPv6 addresses for SIP
sip-proxy          IN     A      198.51.100.19
sip-proxy          IN     AAAA   2001:DB8:1000:2000::19

; IPv4 and IPv6 addresses for XMPP
xmpp-gw            IN     A      198.51.100.19
xmpp-gw            IN     AAAA   2001:DB8:1000:2000::19

; DNS SRV and NAPTR for STUN / TURN
_stun._udp  IN SRV    0 1 3467 turn-server.falcot.com.
_turn._udp  IN SRV    0 1 3467 turn-server.falcot.com.
@           IN NAPTR  10 0 "s" "RELAY:turn.udp" "" _turn._udp.falcot.com.

; DNS SRV and NAPTR records for SIP
_sips._tcp  IN SRV    0 1 5061 sip-proxy.falcot.com.
@           IN NAPTR  10 0 "s" "SIPS+D2T" "" _sips._tcp.falcot.com.

; DNS SRV records for XMPP Server and Client modes:
_xmpp-client._tcp  IN     SRV    5 0 5222 xmpp-gw.falcot.com.
_xmpp-server._tcp  IN     SRV    5 0 5269 xmpp-gw.falcot.com.
```

11.8.2. TURN Server

TURN is a service that helps clients behind NAT routers and firewalls to discover the most efficient way to communicate with other clients and to relay the media streams if no direct media path can be found. It is highly recommended that the TURN server is installed before any of the other RTC services are offered to end users.

TURN and the related ICE protocol are open standards. To benefit from these protocols, maximizing connectivity and minimizing user frustration, it is important to ensure that all client software supports ICE and TURN.

For the ICE algorithm to work effectively, the server must have two public IPv4 addresses.

Install the *coturn* package and edit the `/etc/turnserver.conf` configuration file. By default, a SQLite database is configured in /var/db/turndb for user account settings, but PostgreSQL,

MySQL or Redis can be set up instead if preferred. The most important thing to do is insert the IP addresses of the server.

The server can be started running `/usr/bin/turnserver`. We want the server to be an an automatically started system service, so we edit the `/etc/default/coturn` file like this:

```
#
# Uncomment it if you want to have the turnserver running as
# an automatic system service daemon
#
TURNSERVER_ENABLED=1
```

By default, the TURN server uses anonymous access. We have to add the users we want to use:

```
# turnadmin -a -u roland -p secret_password -r falcot.com
# turnadmin -A -u admin -p secret_password
```

We use the argument -a to add a normal user and -A to add an admin user.

11.8.3. SIP Proxy Server

A SIP proxy server manages the incoming and outgoing SIP connections between other organizations, SIP trunking providers, SIP PBXes such as Asterisk, SIP phones, SIP-based softphones and WebRTC applications.

It is strongly recommended to install and configure the SIP proxy before attempting a SIP PBX setup. The SIP proxy normalizes a lot of the traffic reaching the PBX and provides greater connectivity and resilience.

Install the SIP proxy

Install the *kamailio* package and the package for the database backend, the Falcot administrators chose MySQL, so they install *mariadb-server*. `/etc/kamailio/kamctlrc` is the configuration file for the control tools `kamctl` and `kamdbctl`. You need to edit and set the SIP_DOMAIN to your SIP service domain and set the DBENGINE to MySQL, another database backend can be used.

```
[...]
## your SIP domain
SIP_DOMAIN=sip.falcot.com

## chrooted directory
# $CHROOT_DIR="/path/to/chrooted/directory"

## database type: MYSQL, PGSQL, ORACLE, DB_BERKELEY, DBTEXT, or SQLITE
# by default none is loaded
#
# If you want to setup a database with kamdbctl, you must at least specify
# this parameter.
```

```
DBENGINE=MYSQL
[...]
```

Now we focus on the configuration file `/etc/kamailio/kamailio.cfg`. Falcot needs user authentication and persistent user location, so they add the following #!define directives at the top of that file:

```
#!KAMAILIO
#
# Kamailio (OpenSER) SIP Server v5.2 - default configuration script
#     - web: https://www.kamailio.org
#     - git: https://github.com/kamailio/kamailio
#!define WITH_MYSQL
#!define WITH_AUTH
#!define WITH_USRLOCDB
[...]
```

Kamailio needs a database structure that we can create running `kamdbctl create` as root.

Finally, we can add some users with `kamctl`.

```
# kamctl add roland secret_password
```

Once everything is properly configured you can start or restart the service with `systemctl restart kamailio`, you can connect with a SIP client providing the ip address and the port (5090 is the default port). The users have the following id: *roland@sip.falcot.com*, and they can login using a client (see section 13.10, "Real-Time Communications software" page 397)

11.8.4. XMPP Server

An XMPP server manages connectivity between local XMPP users and XMPP users in other domains on the public Internet.

VOCABULARY
XMPP or Jabber?
XMPP is sometimes referred to as Jabber. In fact, Jabber is a trademark and XMPP is the official name of the standard.

Prosody is a popular XMPP server that operates reliably on Debian servers.

Install the XMPP server

Install the *prosody* package.

Review the `/etc/prosody/prosody.cfg.lua` configuration file. The most important thing to do is insert JIDs of the users who are permitted to manage the server.

```
admins = { "joe@falcot.com" }
```

An individual configuration file is also needed for each domain. Copy the sample from `/etc/prosody/conf.avail/example.com.cfg.lua` and use it as a starting point. Here is falcot.com.cfg.lua:

```
VirtualHost "falcot.com"
        enabled = true
        ssl = {
                key = "/etc/ssl/private/falcot.com-key.pem";
                certificate = "/etc/ssl/public/falcot.com.pem";
                }

-- Set up a MUC (multi-user chat) room server on conference.example.com:
Component "conference.falcot.com" "muc"
```

To enable the domain, there must be a symlink from `/etc/prosody/conf.d/`. Create it that way:

```
# ln -s /etc/prosody/conf.avail/falcot.com.cfg.lua /etc/prosody/conf.d/
```

Restart the service to use the new configuration.

Managing the XMPP server

Some management operations can be performed using the prosodyctl command line utility. For example, to add the administrator account specified in `/etc/prosody/prosody.cfg.lua`:

```
# prosodyctl adduser joe@falcot.com
```

See the Prosody online documentation[4] for more details about how to customize the configuration.

11.8.5. Running services on port 443

Some administrators prefer to run all of their RTC services on port 443. This helps users to connect from remote locations such as hotels and airports where other ports may be blocked or Internet traffic is routed through HTTP proxy servers.

To use this strategy, each service (SIP, XMPP and TURN) needs a different IP address. All the services can still be on the same host as Linux supports multiple IP addresses on a single host. The port number, 443, must be specified in the configuration files for each process and also in the DNS SRV records.

[4] https://prosody.im/doc/configure

11.8.6. Adding WebRTC

Falcot wants to let customers make phone calls directly from the web site. The Falcot administrators also want to use WebRTC as part of their disaster recovery plan, so staff can use web browsers at home to log in to the company phone system and work normally in an emergency.

IN PRACTICE

Try WebRTC

If you have not tried WebRTC before, there are various sites that give an online demonstration and test facilities.

➨ `https://www.sip5060.net/test-calls`

WebRTC is a rapidly evolving technology and it is essential to use packages from the *Testing* distribution. Another option is to compile the software.

WebRTC uses a simple API to provide browsers and mobile applications with RTC, it is free software and it is being developed by Google.

➨ `https://webrtc.org`

A very flexible approach is using GStreamer's WebRTC implementation. It enables pipeline-based multimedia applications, which allows developing interesting and highly efficient applications. A good starting point is the following demo by Centricular, the main company that is developing it:

➨ `https://github.com/centricular/gstwebrtc-demos`

More advanced click-to-call web sites typically use server-side scripting to generate the config.js file dynamically. The DruCall[5] source code demonstrates how to do this with PHP.

This chapter sampled only a fraction of the available server software; however, most of the common network services were described. Now it is time for an even more technical chapter: we'll go into deeper detail for some concepts, describe massive deployments and virtualization.

[5] `https://www.drupal.org/project/drucall`



Keywords

RAID
LVM
FAI
Preseeding
Monitoring
Virtualization
Xen
LXC



Chapter

12

Advanced Administration

Contents

RAID and LVM 328 Virtualization 349 Automated Installation 365 Monitoring 371

This chapter revisits some aspects we already described, with a different perspective: instead of installing one single computer, we will study mass-deployment systems; instead of creating RAID or LVM volumes at install time, we'll learn to do it by hand so we can later revise our initial choices. Finally, we will discuss monitoring tools and virtualization techniques. As a consequence, this chapter is more particularly targeting professional administrators, and focuses somewhat less on individuals responsible for their home network.

chapter 4, "Installation" page 52 presented these technologies from the point of view of the installer, and how it integrated them to make their deployment easy from the start. After the initial installation, an administrator must be able to handle evolving storage space needs without having to resort to an expensive reinstallation. They must therefore understand the required tools for manipulating RAID and LVM volumes.

RAID and LVM are both techniques to abstract the mounted volumes from their physical counterparts (actual hard-disk drives or partitions thereof); the former ensures the security and availability of the data in case of hardware failure by introducing redundancy, the latter makes volume management more flexible and independent of the actual size of the underlying disks. In both cases, the system ends up with new block devices, which can be used to create filesystems or swap space, without necessarily having them mapped to one physical disk. RAID and LVM come from quite different backgrounds, but their functionality can overlap somewhat, which is why they are often mentioned together.

PERSPECTIVE

Btrfs combines LVM and RAID

While LVM and RAID are two distinct kernel subsystems that come between the disk block devices and their filesystems, *btrfs* is a filesystem, initially developed at Oracle, that purports to combine the featuresets of LVM and RAID and much more.

➨ `https://btrfs.wiki.kernel.org/index.php/Main_Page`

Among the noteworthy features are the ability to take a snapshot of a filesystem tree at any point in time. This snapshot copy doesn't initially use any disk space, the data only being duplicated when one of the copies is modified. The filesystem also handles transparent compression of files, and checksums ensure the integrity of all stored data.

In both the RAID and LVM cases, the kernel provides a block device file, similar to the ones corresponding to a hard disk drive or a partition. When an application, or another part of the kernel, requires access to a block of such a device, the appropriate subsystem routes the block to the relevant physical layer. Depending on the configuration, this block can be stored on one or several physical disks, and its physical location may not be directly correlated to the location of the block in the logical device.

12.1.1. Software RAID

RAID means *Redundant Array of Independent Disks*. The goal of this system is to prevent data loss and ensure availability in case of hard disk failure. The general principle is quite simple: data are stored on several physical disks instead of only one, with a configurable level of redundancy. Depending on this amount of redundancy, and even in the event of an unexpected disk failure, data can be losslessly reconstructed from the remaining disks.

CULTURE

Independent **or** *inexpensive*?

The I in RAID initially stood for *inexpensive*, because RAID allowed a drastic increase in data safety without requiring investing in expensive high-end disks. Probably due to image concerns, however, it is now more customarily considered to stand for *independent*, which doesn't have the unsavory flavor of cheapness.

RAID can be implemented either by dedicated hardware (RAID modules integrated into SCSI or SATA controller cards) or by software abstraction (the kernel). Whether hardware or software, a RAID system with enough redundancy can transparently stay operational when a disk fails; the upper layers of the stack (applications) can even keep accessing the data in spite of the failure. Of course, this "degraded mode" can have an impact on performance, and redundancy is reduced, so a further disk failure can lead to data loss. In practice, therefore, one will strive to only stay in this degraded mode for as long as it takes to replace the failed disk. Once the new disk is in place, the RAID system can reconstruct the required data so as to return to a safe mode. The applications won't notice anything, apart from potentially reduced access speed, while the array is in degraded mode or during the reconstruction phase.

When RAID is implemented by hardware, its configuration generally happens within the BIOS setup tool, and the kernel will consider a RAID array as a single disk, which will work as a standard physical disk, although the device name may be different (depending on the driver).

We only focus on software RAID in this book.

Different RAID Levels

RAID is actually not a single system, but a range of systems identified by their levels; the levels differ by their layout and the amount of redundancy they provide. The more redundant, the more failure-proof, since the system will be able to keep working with more failed disks. The counterpart is that the usable space shrinks for a given set of disks; seen the other way, more disks will be needed to store a given amount of data.

Linear RAID Even though the kernel's RAID subsystem allows creating "linear RAID", this is not proper RAID, since this setup doesn't involve any redundancy. The kernel merely aggregates several disks end-to-end and provides the resulting aggregated volume as one virtual disk (one block device). That is about its only function. This setup is rarely used by itself (see later for the exceptions), especially since the lack of redundancy means that one disk failing makes the whole aggregate, and therefore all the data, unavailable.

RAID-0 This level doesn't provide any redundancy either, but disks aren't simply stuck on end one after another: they are divided in *stripes*, and the blocks on the virtual device are stored on stripes on alternating physical disks. In a two-disk RAID-0 setup, for instance, even-numbered blocks of the virtual device will be stored on the first physical disk, while odd-numbered blocks will end up on the second physical disk.

This system doesn't aim at increasing reliability, since (as in the linear case) the availability of all the data is jeopardized as soon as one disk fails, but at increasing performance:

during sequential access to large amounts of contiguous data, the kernel will be able to read from both disks (or write to them) in parallel, which increases the data transfer rate. The disks are utilized entirely by the RAID device, so they should have the same size not to lose performance.

RAID-0 use is shrinking, its niche being filled by LVM (see later).

RAID-1 This level, also known as "RAID mirroring", is both the simplest and the most widely used setup. In its standard form, it uses two physical disks of the same size, and provides a logical volume of the same size again. Data are stored identically on both disks, hence the "mirror" nickname. When one disk fails, the data is still available on the other. For really critical data, RAID-1 can of course be set up on more than two disks, with a direct impact on the ratio of hardware cost versus available payload space.

NOTE

Disks and cluster sizes

If two disks of different sizes are set up in a mirror, the bigger one will not be fully used, since it will contain the same data as the smallest one and nothing more. The useful available space provided by a RAID-1 volume therefore matches the size of the smallest disk in the array. This still holds for RAID volumes with a higher RAID level, even though redundancy is stored differently.

It is therefore important, when setting up RAID arrays (except for RAID-0 and "linear RAID"), to only assemble disks of identical, or very close, sizes, to avoid wasting resources.

NOTE

Spare disks

RAID levels that include redundancy allow assigning more disks than required to an array. The extra disks are used as spares when one of the main disks fails. For instance, in a mirror of two disks plus one spare, if one of the first two disks fails, the kernel will automatically (and immediately) reconstruct the mirror using the spare disk, so that redundancy stays assured after the reconstruction time. This can be used as another kind of safeguard for critical data.

One would be forgiven for wondering how this is better than simply mirroring on three disks to start with. The advantage of the "spare disk" configuration is that the spare disk can be shared across several RAID volumes. For instance, one can have three mirrored volumes, with redundancy assured even in the event of one disk failure, with only seven disks (three pairs, plus one shared spare), instead of the nine disks that would be required by three triplets.

This RAID level, although expensive (since only half of the physical storage space, at best, is useful), is widely used in practice. It is simple to understand, and it allows very simple backups: since both disks have identical contents, one of them can be temporarily extracted with no impact on the working system. Read performance is often increased since the kernel can read half of the data on each disk in parallel, while write performance isn't too severely degraded. In case of a RAID-1 array of N disks, the data stays available even with N-1 disk failures.

RAID-4 This RAID level, not widely used, uses N disks to store useful data, and an extra disk to store redundancy information. If that disk fails, the system can reconstruct its contents

from the other N. If one of the N data disks fails, the remaining N-1 combined with the "parity" disk contain enough information to reconstruct the required data.

RAID-4 isn't too expensive since it only involves a one-in-N increase in costs and has no noticeable impact on read performance, but writes are slowed down. Furthermore, since a write to any of the N disks also involves a write to the parity disk, the latter sees many more writes than the former, and its lifespan can shorten dramatically as a consequence. Data on a RAID-4 array is safe only up to one failed disk (of the N+1).

RAID-5 RAID-5 addresses the asymmetry issue of RAID-4: parity blocks are spread over all of the N+1 disks, with no single disk having a particular role.

Read and write performance are identical to RAID-4. Here again, the system stays functional with up to one failed disk (of the N+1), but no more.

RAID-6 RAID-6 can be considered an extension of RAID-5, where each series of N blocks involves two redundancy blocks, and each such series of N+2 blocks is spread over N+2 disks.

This RAID level is slightly more expensive than the previous two, but it brings some extra safety since up to two drives (of the N+2) can fail without compromising data availability. The counterpart is that write operations now involve writing one data block and two redundancy blocks, which makes them even slower.

RAID-1+0 This isn't strictly speaking, a RAID level, but a stacking of two RAID groupings. Starting from 2×N disks, one first sets them up by pairs into N RAID-1 volumes; these N volumes are then aggregated into one, either by "linear RAID" or (increasingly) by LVM. This last case goes farther than pure RAID, but there is no problem with that.

RAID-1+0 can survive multiple disk failures: up to N in the 2×N array described above, provided that at least one disk keeps working in each of the RAID-1 pairs.

GOING FURTHER

RAID-10

RAID-10 is generally considered a synonym of RAID-1+0, but a Linux specificity makes it actually a generalization. This setup allows a system where each block is stored on two different disks, even with an odd number of disks, the copies being spread out along a configurable model.

Performances will vary depending on the chosen repartition model and redundancy level, and of the workload of the logical volume.

Obviously, the RAID level will be chosen according to the constraints and requirements of each application. Note that a single computer can have several distinct RAID arrays with different configurations.

Setting up RAID

Setting up RAID volumes requires the *mdadm* package; it provides the `mdadm` command, which allows creating and manipulating RAID arrays, as well as scripts and tools integrating it to the rest of the system, including the monitoring system.

Our example will be a server with a number of disks, some of which are already used, the rest being available to setup RAID. We initially have the following disks and partitions:

- the `sdb` disk, 4 GB, is entirely available;
- the `sdc` disk, 4 GB, is also entirely available;
- on the `sdd` disk, only partition `sdd2` (about 4 GB) is available;
- finally, a `sde` disk, still 4 GB, entirely available.

Identifying existing RAID volumes

NOTE

The `/proc/mdstat` file lists existing volumes and their states. When creating a new RAID volume, care should be taken not to name it the same as an existing volume.

We're going to use these physical elements to build two volumes, one RAID-0 and one mirror (RAID-1). Let's start with the RAID-0 volume:

```
# mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/sdb /dev/sdc
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
# mdadm --query /dev/md0
/dev/md0: 8.00GiB raid0 2 devices, 0 spares. Use mdadm --detail for more detail.
# mdadm --detail /dev/md0
/dev/md0:
           Version : 1.2
     Creation Time : Tue Jun 25 08:47:49 2019
        Raid Level : raid0
        Array Size : 8378368 (7.99 GiB 8.58 GB)
      Raid Devices : 2
     Total Devices : 2
       Persistence : Superblock is persistent

       Update Time : Tue Jun 25 08:47:49 2019
             State : clean
    Active Devices : 2
   Working Devices : 2
    Failed Devices : 0
     Spare Devices : 0

        Chunk Size : 512K

Consistency Policy : none

              Name : mirwiz:0  (local to host debian)
              UUID : 146e104f:66ccc06d:71c262d7:9af1fbc7
            Events : 0

    Number   Major   Minor   RaidDevice State
       0       8       32        0      active sync   /dev/sdb
       1       8       48        1      active sync   /dev/sdc
```

```
# mkfs.ext4 /dev/md0
mke2fs 1.44.5 (15-Dec-2018)
Discarding device blocks: done
Creating filesystem with 2094592 4k blocks and 524288 inodes
Filesystem UUID: 413c3dff-ab5e-44e7-ad34-cf1a029cfe98
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

# mkdir /srv/raid-0
# mount /dev/md0 /srv/raid-0
# df -h /srv/raid-0
Filesystem      Size  Used Avail Use% Mounted on
/dev/md0        7.9G   36M  7.4G   1% /srv/raid-0
```

The `mdadm --create` command requires several parameters: the name of the volume to create (`/dev/md*`, with MD standing for *Multiple Device*), the RAID level, the number of disks (which is compulsory despite being mostly meaningful only with RAID-1 and above), and the physical drives to use. Once the device is created, we can use it like we'd use a normal partition, create a filesystem on it, mount that filesystem, and so on. Note that our creation of a RAID-0 volume on `md0` is nothing but coincidence, and the numbering of the array doesn't need to be correlated to the chosen amount of redundancy. It is also possible to create named RAID arrays, by giving `mdadm` parameters such as `/dev/md/linear` instead of `/dev/md0`.

Creation of a RAID-1 follows a similar fashion, the differences only being noticeable after the creation:

```
# mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/sdd2 /dev/sde
mdadm: Note: this array has metadata at the start and
    may not be suitable as a boot device.  If you plan to
    store '/boot' on this device please ensure that
    your boot-loader understands md/v1.x metadata, or use
    --metadata=0.90
mdadm: largest drive (/dev/sdd2) exceeds size (4192192K) by more than 1%
Continue creating array? y
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md1 started.
# mdadm --query /dev/md1
/dev/md1: 4.00GiB raid1 2 devices, 0 spares. Use mdadm --detail for more detail.
# mdadm --detail /dev/md1
/dev/md1:
           Version : 1.2
     Creation Time : Tue Jun 25 10:21:22 2019
        Raid Level : raid1
        Array Size : 4189184 (4.00 GiB 4.29 GB)
```

```
     Used Dev Size : 4189184 (4.00 GiB 4.29 GB)
      Raid Devices : 2
     Total Devices : 2
       Persistence : Superblock is persistent

       Update Time : Tue Jun 25 10:22:03 2019
             State : clean, resyncing
    Active Devices : 2
   Working Devices : 2
    Failed Devices : 0
     Spare Devices : 0

Consistency Policy : resync

     Resync Status : 93% complete

              Name : mirwiz:1  (local to host debian)
              UUID : 7d123734:9677b7d6:72194f7d:9050771c
            Events : 16

    Number   Major   Minor   RaidDevice State
       0       8       64        0      active sync   /dev/sdd2
       1       8       80        1      active sync   /dev/sde
# mdadm --detail /dev/md1
/dev/md1:
[...]
          State : clean
[...]
```

TIP **RAID, disks and partitions**

As illustrated by our example, RAID devices can be constructed out of disk partitions, and do not require full disks.

A few remarks are in order. First, `mdadm` notices that the physical elements have different sizes; since this implies that some space will be lost on the bigger element, a confirmation is required.

More importantly, note the state of the mirror. The normal state of a RAID mirror is that both disks have exactly the same contents. However, nothing guarantees this is the case when the volume is first created. The RAID subsystem will therefore provide that guarantee itself, and there will be a synchronization phase as soon as the RAID device is created. After some time (the exact amount will depend on the actual size of the disks...), the RAID array switches to the "active" or "clean" state. Note that during this reconstruction phase, the mirror is in a degraded mode, and redundancy isn't assured. A disk failing during that risk window could lead to losing all the data. Large amounts of critical data, however, are rarely stored on a freshly created RAID array before its initial synchronization. Note that even in degraded mode, the `/dev/md1` is usable, and a filesystem can be created on it, as well as some data copied on it.

TIP

Starting a mirror in degraded mode

Sometimes two disks are not immediately available when one wants to start a RAID-1 mirror, for instance because one of the disks one plans to include is already used to store the data one wants to move to the array. In such circumstances, it is possible to deliberately create a degraded RAID-1 array by passing `missing` instead of a device file as one of the arguments to `mdadm`. Once the data have been copied to the "mirror", the old disk can be added to the array. A synchronization will then take place, giving us the redundancy that was wanted in the first place.

TIP

Setting up a mirror without synchronization

RAID-1 volumes are often created to be used as a new disk, often considered blank. The actual initial contents of the disk is therefore not very relevant, since one only needs to know that the data written after the creation of the volume, in particular the filesystem, can be accessed later.

One might therefore wonder about the point of synchronizing both disks at creation time. Why care whether the contents are identical on zones of the volume that we know will only be read after we have written to them?

Fortunately, this synchronization phase can be avoided by passing the `--assume-clean` option to `mdadm`. However, this option can lead to surprises in cases where the initial data will be read (for instance if a filesystem is already present on the physical disks), which is why it isn't enabled by default.

Now let's see what happens when one of the elements of the RAID-1 array fails. `mdadm`, in particular its --fail option, allows simulating such a disk failure:

```
# mdadm /dev/md1 --fail /dev/sde
mdadm: set /dev/sde faulty in /dev/md1
# mdadm --detail /dev/md1
/dev/md1:
[...]
       Update Time : Tue Jun 25 11:03:44 2019
             State : clean, degraded
    Active Devices : 1
   Working Devices : 1
    Failed Devices : 1
     Spare Devices : 0

Consistency Policy : resync

              Name : mirwiz:1  (local to host debian)
              UUID : 7d123734:9677b7d6:72194f7d:9050771c
            Events : 20

    Number   Major   Minor   RaidDevice State
       -       0        0        0      removed
       1       8       80        1      active sync   /dev/sdd2

       0       8       64        -      faulty   /dev/sde
```

The contents of the volume are still accessible (and, if it is mounted, the applications don't notice a thing), but the data safety isn't assured anymore: should the `sdd` disk fail in turn, the data would be lost. We want to avoid that risk, so we'll replace the failed disk with a new one, `sdf`:

```
# mdadm /dev/md1 --add /dev/sdf
mdadm: added /dev/sdf
# mdadm --detail /dev/md1
/dev/md1:
[...]
      Raid Devices : 2
     Total Devices : 3
       Persistence : Superblock is persistent

       Update Time : Tue Jun 25 11:09:42 2019
             State : clean, degraded, recovering
    Active Devices : 1
   Working Devices : 2
    Failed Devices : 1
     Spare Devices : 1

Consistency Policy : resync

    Rebuild Status : 27% complete

              Name : mirwiz:1  (local to host debian)
              UUID : 7d123734:9677b7d6:72194f7d:9050771c
            Events : 26

    Number   Major   Minor   RaidDevice State
       2       8       96        0      spare rebuilding   /dev/sdf
       1       8       80        1      active sync   /dev/sdd2

       0       8       64        -      faulty   /dev/sde
# [...]
[...]
# mdadm --detail /dev/md1
/dev/md1:
[...]
       Update Time : Tue Jun 25 11:10:47 2019
             State : clean
    Active Devices : 2
   Working Devices : 2
    Failed Devices : 1
     Spare Devices : 0

Consistency Policy : resync

              Name : mirwiz:1  (local to host debian)
              UUID : 7d123734:9677b7d6:72194f7d:9050771c
            Events : 39
```

```
    Number   Major   Minor   RaidDevice State
       2       8       96        0      active sync   /dev/sdd2
       1       8       80        1      active sync   /dev/sdf

       0       8       64        -      faulty   /dev/sde
```

Here again, the kernel automatically triggers a reconstruction phase during which the volume, although still accessible, is in a degraded mode. Once the reconstruction is over, the RAID array is back to a normal state. One can then tell the system that the `sde` disk is about to be removed from the array, so as to end up with a classical RAID mirror on two disks:

```
# mdadm /dev/md1 --remove /dev/sde
mdadm: hot removed /dev/sde from /dev/md1
# mdadm --detail /dev/md1
/dev/md1:
[...]
    Number   Major   Minor   RaidDevice State
       2       8       96        0      active sync   /dev/sdd2
       1       8       80        1      active sync   /dev/sdf
```

From then on, the drive can be physically removed when the server is next switched off, or even hot-removed when the hardware configuration allows hot-swap. Such configurations include some SCSI controllers, most SATA disks, and external drives operating on USB or Firewire.

Backing up the Configuration

Most of the meta-data concerning RAID volumes are saved directly on the disks that make up these arrays, so that the kernel can detect the arrays and their components and assemble them automatically when the system starts up. However, backing up this configuration is encouraged, because this detection isn't fail-proof, and it is only expected that it will fail precisely in sensitive circumstances. In our example, if the `sde` disk failure had been real (instead of simulated) and the system had been restarted without removing this `sde` disk, this disk could start working again due to having been probed during the reboot. The kernel would then have three physical elements, each claiming to contain half of the same RAID volume. Another source of confusion can come when RAID volumes from two servers are consolidated onto one server only. If these arrays were running normally before the disks were moved, the kernel would be able to detect and reassemble the pairs properly; but if the moved disks had been aggregated into an `md1` on the old server, and the new server already has an `md1`, one of the mirrors would be renamed.

Backing up the configuration is therefore important, if only for reference. The standard way to do it is by editing the `/etc/mdadm/mdadm.conf` file, an example of which is listed here:

Example 12.1 *`mdadm` configuration file*

```
# mdadm.conf
```

```
#
# !NB! Run update-initramfs -u after updating this file.
# !NB! This will ensure that initramfs has an uptodate copy.
#
# Please refer to mdadm.conf(5) for information about this file.
#

# by default (built-in), scan all partitions (/proc/partitions) and all
# containers for MD superblocks. alternatively, specify devices to scan, using
# wildcards if desired.
DEVICE /dev/sd*

# auto-create devices with Debian standard permissions
CREATE owner=root group=disk mode=0660 auto=yes

# automatically tag new arrays as belonging to the local system
HOMEHOST <system>

# instruct the monitoring daemon where to send mail alerts
MAILADDR root

# definitions of existing MD arrays
ARRAY /dev/md0 metadata=1.2 name=mirwiz:0 UUID=146e104f:66ccc06d:71c262d7:9af1fbc7
ARRAY /dev/md1 metadata=1.2 name=mirwiz:1 UUID=7d123734:9677b7d6:72194f7d:9050771c

# This configuration was auto-generated on Tue, 25 Jun 2019 07:54:35 -0400 by mkconf
```

One of the most useful details is the DEVICE option, which lists the devices where the system will automatically look for components of RAID volumes at start-up time. In our example, we replaced the default value, partitions containers, with an explicit list of device files, since we chose to use entire disks and not only partitions, for some volumes.

The last two lines in our example are those allowing the kernel to safely pick which volume number to assign to which array. The metadata stored on the disks themselves are enough to re-assemble the volumes, but not to determine the volume number (and the matching `/dev/md*` device name).

Fortunately, these lines can be generated automatically:

```
# mdadm --misc --detail --brief /dev/md?
ARRAY /dev/md0 metadata=1.2 name=mirwiz:0 UUID=146e104f:66ccc06d:71c262d7:9af1fbc7
ARRAY /dev/md1 metadata=1.2 name=mirwiz:1 UUID=7d123734:9677b7d6:72194f7d:9050771c
```

The contents of these last two lines doesn't depend on the list of disks included in the volume. It is therefore not necessary to regenerate these lines when replacing a failed disk with a new one. On the other hand, care must be taken to update the file when creating or deleting a RAID array.

12.1.2. LVM

LVM, the *Logical Volume Manager*, is another approach to abstracting logical volumes from their physical supports, which focuses on increasing flexibility rather than increasing reliability. LVM allows changing a logical volume transparently as far as the applications are concerned; for instance, it is possible to add new disks, migrate the data to them, and remove the old disks, without unmounting the volume.

LVM Concepts

This flexibility is attained by a level of abstraction involving three concepts.

First, the PV (*Physical Volume*) is the entity closest to the hardware: it can be partitions on a disk, or a full disk, or even any other block device (including, for instance, a RAID array). Note that when a physical element is set up to be a PV for LVM, it should only be accessed via LVM, otherwise the system will get confused.

A number of PVs can be clustered in a VG (*Volume Group*), which can be compared to disks both virtual and extensible. VGs are abstract, and don't appear in a device file in the `/dev` hierarchy, so there is no risk of using them directly.

The third kind of object is the LV (*Logical Volume*), which is a chunk of a VG; if we keep the VG-as-disk analogy, the LV compares to a partition. The LV appears as a block device with an entry in `/dev`, and it can be used as any other physical partition can be (most commonly, to host a filesystem or swap space).

The important thing is that the splitting of a VG into LVs is entirely independent of its physical components (the PVs). A VG with only a single physical component (a disk for instance) can be split into a dozen logical volumes; similarly, a VG can use several physical disks and appear as a single large logical volume. The only constraint, obviously, is that the total size allocated to LVs can't be bigger than the total capacity of the PVs in the volume group.

It often makes sense, however, to have some kind of homogeneity among the physical components of a VG, and to split the VG into logical volumes that will have similar usage patterns. For instance, if the available hardware includes fast disks and slower disks, the fast ones could be clustered into one VG and the slower ones into another; chunks of the first one can then be assigned to applications requiring fast data access, while the second one will be kept for less demanding tasks.

In any case, keep in mind that an LV isn't particularly attached to any one PV. It is possible to influence where the data from an LV are physically stored, but this possibility isn't required for day-to-day use. On the contrary: when the set of physical components of a VG evolves, the physical storage locations corresponding to a particular LV can be migrated across disks (while staying within the PVs assigned to the VG, of course).

Setting up LVM

Let us now follow, step by step, the process of setting up LVM for a typical use case: we want to simplify a complex storage situation. Such a situation usually happens after some long and convoluted history of accumulated temporary measures. For the purposes of illustration, we'll consider a server where the storage needs have changed over time, ending up in a maze of available partitions split over several partially used disks. In more concrete terms, the following partitions are available:

- on the `sdb` disk, a `sdb2` partition, 4 GB;
- on the `sdc` disk, a `sdc3` partition, 3 GB;
- the `sdd` disk, 4 GB, is fully available;
- on the `sdf` disk, a `sdf1` partition, 4 GB; and a `sdf2` partition, 5 GB.

In addition, let's assume that disks `sdb` and `sdf` are faster than the other two.

Our goal is to set up three logical volumes for three different applications: a file server requiring 5 GB of storage space, a database (1 GB) and some space for back-ups (12 GB). The first two need good performance, but back-ups are less critical in terms of access speed. All these constraints prevent the use of partitions on their own; using LVM can abstract the physical size of the devices, so the only limit is the total available space.

The required tools are in the *lvm2* package and its dependencies. When they're installed, setting up LVM takes three steps, matching the three levels of concepts.

First, we prepare the physical volumes using `pvcreate`:

```
# pvcreate /dev/sdb2
  Physical volume ”/dev/sdb2” successfully created.
# pvdisplay
  ”/dev/sdb2” is a new physical volume of ”4.00 GiB”
  --- NEW Physical volume ---
  PV Name               /dev/sdb2
  VG Name
  PV Size               4.00 GiB
  Allocatable           NO
  PE Size               0
  Total PE              0
  Free PE               0
  Allocated PE          0
  PV UUID               z4Clgk-T5a4-C27o-1P0E-lIAF-OeUM-e7EMwq

# for i in sdc3 sdd sdf1 sdf2 ; do pvcreate /dev/$i ; done
  Physical volume ”/dev/sdc3” successfully created.
  Physical volume ”/dev/sdd” successfully created.
  Physical volume ”/dev/sdf1” successfully created.
  Physical volume ”/dev/sdf2” successfully created.
# pvdisplay -C
  PV         VG Fmt  Attr PSize  PFree
```

```
  /dev/sdb2     lvm2 ---   4.00g  4.00g
  /dev/sdc3     lvm2 ---   3.00g  3.00g
  /dev/sdd      lvm2 ---   4.00g  4.00g
  /dev/sdf1     lvm2 ---   4.00g  4.00g
  /dev/sdf2     lvm2 ---  <5.00g <5.00g
```

So far, so good; note that a PV can be set up on a full disk as well as on individual partitions of it. As shown above, the `pvdisplay` command lists the existing PVs, with two possible output formats.

Now let's assemble these physical elements into VGs using `vgcreate`. We'll gather only PVs from the fast disks into a `vg_critical` VG; the other VG, `vg_normal`, will also include slower elements.

```
# vgcreate vg_critical /dev/sdb2 /dev/sdf1
  Volume group "vg_critical" successfully created
# vgdisplay
  --- Volume group ---
  VG Name               vg_critical
  System ID
  Format                lvm2
  Metadata Areas        2
  Metadata Sequence No  1
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                0
  Open LV               0
  Max PV                0
  Cur PV                2
  Act PV                2
  VG Size               7.99 GiB
  PE Size               4.00 MiB
  Total PE              2046
  Alloc PE / Size       0 / 0
  Free  PE / Size       2046 / 7.99 GiB
  VG UUID               wAbBjx-d82B-q7St-0KFf-z40h-w5Mh-uAXkNZ

# vgcreate vg_normal /dev/sdc3 /dev/sdd /dev/sdf2
  Volume group "vg_normal" successfully created
# vgdisplay -C
  VG          #PV #LV #SN Attr   VSize   VFree
  vg_critical   2   0   0 wz--n-   7.99g   7.99g
  vg_normal     3   0   0 wz--n- <11.99g <11.99g
```

Here again, commands are rather straightforward (and `vgdisplay` proposes two output formats). Note that it is quite possible to use two partitions of the same physical disk into two different VGs. Note also that we used a `vg_` prefix to name our VGs, but it is nothing more than a convention.

We now have two "virtual disks", sized about 8 GB and 12 GB respectively. Let's now carve them up into "virtual partitions" (LVs). This involves the `lvcreate` command, and a slightly more complex syntax:

```
# lvdisplay
# lvcreate -n lv_files -L 5G vg_critical
  Logical volume "lv_files" created.
# lvdisplay
  --- Logical volume ---
  LV Path                /dev/vg_critical/lv_files
  LV Name                lv_files
  VG Name                vg_critical
  LV UUID                W6XT08-iBBx-Nrw2-f8F2-r2y4-Ltds-UrKogV
  LV Write Access        read/write
  LV Creation host, time debian, 2019-11-30 22:45:46 -0500
  LV Status              available
  # open                 0
  LV Size                5.00 GiB
  Current LE             1280
  Segments               2
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           254:0

# lvcreate -n lv_base -L 1G vg_critical
  Logical volume "lv_base" created.
# lvcreate -n lv_backups -L 11.98G vg_normal
  Rounding up size to full physical extent 11.98 GiB
  Logical volume "lv_backups" created.
# lvdisplay -C
  LV         VG          Attr     LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync
      ➥  Convert
  lv_base    vg_critical -wi-a---  1.00g
  lv_files   vg_critical -wi-a---  5.00g
  lv_backups vg_normal   -wi-a--- 11.98g
```

Two parameters are required when creating logical volumes; they must be passed to the `lvcreate` as options. The name of the LV to be created is specified with the `-n` option, and its size is generally given using the `-L` option. We also need to tell the command what VG to operate on, of course, hence the last parameter on the command line.

GOING FURTHER

lvcreate options

The `lvcreate` command has several options to allow tweaking how the LV is created.

Let's first describe the `-l` option, with which the LV's size can be given as a number of blocks (as opposed to the "human" units we used above). These blocks (called PEs, *physical extents*, in LVM terms) are contiguous units of storage space in PVs, and they can't be split across LVs. When one wants to define storage space for an

LV with some precision, for instance to use the full available space, the `-l` option will probably be preferred over `-L`.

It is also possible to hint at the physical location of an LV, so that its extents are stored on a particular PV (while staying within the ones assigned to the VG, of course). Since we know that `sdb` is faster than `sdf`, we may want to store the `lv_base` there if we want to give an advantage to the database server compared to the file server. The command line becomes: `lvcreate -n lv_base -L 1G vg_critical /dev/sdb2`. Note that this command can fail if the PV doesn't have enough free extents. In our example, we would probably have to create `lv_base` before `lv_files` to avoid this situation – or free up some space on `sdb2` with the `pvmove` command.

Logical volumes, once created, end up as block device files in `/dev/mapper/`:

```
# ls -l /dev/mapper
total 0
crw------- 1 root root 10, 236 Jun 10 16:52 control
lrwxrwxrwx 1 root root       7 Jun 10 17:05 vg_critical-lv_base -> ../dm-1
lrwxrwxrwx 1 root root       7 Jun 10 17:05 vg_critical-lv_files -> ../dm-0
lrwxrwxrwx 1 root root       7 Jun 10 17:05 vg_normal-lv_backups -> ../dm-2
# ls -l /dev/dm-*
brw-rw---T 1 root disk 253, 0 Jun 10 17:05 /dev/dm-0
brw-rw---- 1 root disk 253, 1 Jun 10 17:05 /dev/dm-1
brw-rw---- 1 root disk 253, 2 Jun 10 17:05 /dev/dm-2
```

NOTE

Auto-detecting LVM volumes

When the computer boots, the `lvm2-activation` systemd service unit executes `vgchange -aay` to "activate" the volume groups: it scans the available devices; those that have been initialized as physical volumes for LVM are registered into the LVM subsystem, those that belong to volume groups are assembled, and the relevant logical volumes are started and made available. There is therefore no need to edit configuration files when creating or modifying LVM volumes.

Note, however, that the layout of the LVM elements (physical and logical volumes, and volume groups) is backed up in `/etc/lvm/backup`, which can be useful in case of a problem (or just to sneak a peek under the hood).

To make things easier, convenience symbolic links are also created in directories matching the VGs:

```
# ls -l /dev/vg_critical
total 0
lrwxrwxrwx 1 root root 7 Jun 10 17:05 lv_base -> ../dm-1
lrwxrwxrwx 1 root root 7 Jun 10 17:05 lv_files -> ../dm-0
# ls -l /dev/vg_normal
total 0
lrwxrwxrwx 1 root root 7 Jun 10 17:05 lv_backups -> ../dm-2
```

The LVs can then be used exactly like standard partitions:

```
# mkfs.ext4 /dev/vg_normal/lv_backups
mke2fs 1.44.5 (15-Dec-2018)
Discarding device blocks: done
Creating filesystem with 3140608 4k blocks and 786432 inodes
Filesystem UUID: b9e6ed2f-cb37-43e9-87d8-e77568446225
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

# mkdir /srv/backups
# mount /dev/vg_normal/lv_backups /srv/backups
# df -h /srv/backups
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/vg_normal-lv_backups   12G   41M   12G   1% /srv/backups
# [...]
[...]
# cat /etc/fstab
[...]
/dev/vg_critical/lv_base    /srv/base       ext4 defaults 0 2
/dev/vg_critical/lv_files   /srv/files      ext4 defaults 0 2
/dev/vg_normal/lv_backups   /srv/backups    ext4 defaults 0 2
```

From the applications' point of view, the myriad small partitions have now been abstracted into one large 12 GB volume, with a friendlier name.

LVM Over Time

Even though the ability to aggregate partitions or physical disks is convenient, this is not the main advantage brought by LVM. The flexibility it brings is especially noticed as time passes, when needs evolve. In our example, let's assume that new large files must be stored, and that the LV dedicated to the file server is too small to contain them. Since we haven't used the whole space available in `vg_critical`, we can grow `lv_files`. For that purpose, we'll use the `lvresize` command, then `resize2fs` to adapt the filesystem accordingly:

```
# df -h /srv/files/
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/vg_critical-lv_files  4.9G  4.2G  485M  90% /srv/files
# lvdisplay -C vg_critical/lv_files
  LV       VG          Attr     LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync
      ➥ Convert
  lv_files vg_critical -wi-ao-- 5.00g
# vgdisplay -C vg_critical
  VG          #PV #LV #SN Attr   VSize VFree
```

```
  vg_critical   2   2   0 wz--n- 7.99g 1.99g
# lvresize -L 6G vg_critical/lv_files
  Size of logical volume vg_critical/lv_files changed from 5.00 GiB (1280 extents) to
      ➥  6.00 GiB (1536 extents).
  Logical volume vg_critical/lv_files successfully resized.
# lvdisplay -C vg_critical/lv_files
  LV       VG          Attr       LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync
      ➥ Convert
  lv_files vg_critical -wi-ao---- 6.00g
# resize2fs /dev/vg_critical/lv_files
resize2fs 1.44.5 (15-Dec-2018)
Filesystem at /dev/vg_critical/lv_files is mounted on /srv/files; on-line resizing
    ➥ required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/vg_critical/lv_files is now 1572864 (4k) blocks long.

# df -h /srv/files/
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/vg_critical-lv_files  5.9G  4.2G  1.5G  75% /srv/files
```

CAUTION

Resizing filesystems

Not all filesystems can be resized online; resizing a volume can therefore require unmounting the filesystem first and remounting it afterwards. Of course, if one wants to shrink the space allocated to an LV, the filesystem must be shrunk first; the order is reversed when the resizing goes in the other direction: the logical volume must be grown before the filesystem on it. It is rather straightforward, since at no time must the filesystem size be larger than the block device where it resides (whether that device is a physical partition or a logical volume).

The ext3, ext4 and xfs filesystems can be grown online, without unmounting; shrinking requires an unmount. The reiserfs filesystem allows online resizing in both directions. The venerable ext2 allows neither, and always requires unmounting.

We could proceed in a similar fashion to extend the volume hosting the database, only we've reached the VG's available space limit:

```
# df -h /srv/base/
Filesystem                        Size  Used Avail Use% Mounted on
/dev/mapper/vg_critical-lv_base  976M  882M   28M  97% /srv/base
# vgdisplay -C vg_critical
  VG          #PV #LV #SN Attr   VSize VFree
  vg_critical   2   2   0 wz--n- 7.99g 1016.00m
```

No matter, since LVM allows adding physical volumes to existing volume groups. For instance, maybe we've noticed that the `sdb1` partition, which was so far used outside of LVM, only contained archives that could be moved to `lv_backups`. We can now recycle it and integrate it to the volume group, and thereby reclaim some available space. This is the purpose of the `vgextend` command. Of course, the partition must be prepared as a physical volume beforehand. Once the VG has been extended, we can use similar commands as previously to grow the logical volume then the filesystem:

```
# pvcreate /dev/sdb1
  Physical volume "/dev/sdb1" successfully created.
# vgextend vg_critical /dev/sdb1
  Volume group "vg_critical" successfully extended
# vgdisplay -C vg_critical
  VG          #PV #LV #SN Attr   VSize  VFree
  vg_critical   3   2   0 wz--n- <9.99g <1.99g
# [...]
[...]
# df -h /srv/base/
Filesystem                      Size  Used Avail Use% Mounted on
/dev/mapper/vg_critical-lv_base  2.0G  882M  994M  48% /srv/base
```

GOING FURTHER

Advanced LVM

LVM also caters for more advanced uses, where many details can be specified by hand. For instance, an administrator can tweak the size of the blocks that make up physical and logical volumes, as well as their physical layout. It is also possible to move blocks across PVs, for instance, to fine-tune performance or, in a more mundane way, to free a PV when one needs to extract the corresponding physical disk from the VG (whether to affect it to another VG or to remove it from LVM altogether). The manual pages describing the commands are generally clear and detailed. A good entry point is the `lvm(8)` manual page.

12.1.3. RAID or LVM?

RAID and LVM both bring indisputable advantages as soon as one leaves the simple case of a desktop computer with a single hard disk where the usage pattern doesn't change over time. However, RAID and LVM go in two different directions, with diverging goals, and it is legitimate to wonder which one should be adopted. The most appropriate answer will of course depend on current and foreseeable requirements.

There are a few simple cases where the question doesn't really arise. If the requirement is to safeguard data against hardware failures, then obviously RAID will be set up on a redundant array of disks, since LVM doesn't really address this problem. If, on the other hand, the need is for a flexible storage scheme where the volumes are made independent of the physical layout of the disks, RAID doesn't help much and LVM will be the natural choice.

NOTE

If performance matters...

If input/output speed is of the essence, especially in terms of access times, using LVM and/or RAID in one of the many combinations may have some impact on performances, and this may influence decisions as to which to pick. However, these differences in performance are really minor, and will only be measurable in a few use cases. If performance matters, the best gain to be obtained would be to use non-rotating storage media (*solid-state drives* or SSDs); their cost per megabyte is higher than that of standard hard disk drives, and their capacity is usually smaller, but they provide excellent performance for random accesses. If the usage pattern includes many input/output operations scattered all around the filesystem, for instance for databases where complex queries are routinely being run, then the advantage of running them on an SSD far outweigh whatever could be gained by

picking LVM over RAID or the reverse. In these situations, the choice should be determined by other considerations than pure speed, since the performance aspect is most easily handled by using SSDs.

The third notable use case is when one just wants to aggregate two disks into one volume, either for performance reasons or to have a single filesystem that is larger than any of the available disks. This case can be addressed both by a RAID-0 (or even linear-RAID) and by an LVM volume. When in this situation, and barring extra constraints (for instance, keeping in line with the rest of the computers if they only use RAID), the configuration of choice will often be LVM. The initial set up is barely more complex, and that slight increase in complexity more than makes up for the extra flexibility that LVM brings if the requirements change or if new disks need to be added.

Then of course, there is the really interesting use case, where the storage system needs to be made both resistant to hardware failure and flexible when it comes to volume allocation. Neither RAID nor LVM can address both requirements on their own; no matter, this is where we use both at the same time — or rather, one on top of the other. The scheme that has all but become a standard since RAID and LVM have reached maturity is to ensure data redundancy first by grouping disks in a small number of large RAID arrays, and to use these RAID arrays as LVM physical volumes; logical partitions will then be carved from these LVs for filesystems. The selling point of this setup is that when a disk fails, only a small number of RAID arrays will need to be reconstructed, thereby limiting the time spent by the administrator for recovery.

Let's take a concrete example: the public relations department at Falcot Corp needs a workstation for video editing, but the department's budget doesn't allow investing in high-end hardware from the bottom up. A decision is made to favor the hardware that is specific to the graphic nature of the work (monitor and video card), and to stay with generic hardware for storage. However, as is widely known, digital video does have some particular requirements for its storage: the amount of data to store is large, and the throughput rate for reading and writing this data is important for the overall system performance (more than typical access time, for instance). These constraints need to be fulfilled with generic hardware, in this case two 300 GB SATA hard disk drives; the system data must also be made resistant to hardware failure, as well as some of the user data. Edited videoclips must indeed be safe, but video rushes pending editing are less critical, since they're still on the videotapes.

RAID-1 and LVM are combined to satisfy these constraints. The disks are attached to two different SATA controllers to optimize parallel access and reduce the risk of a simultaneous failure, and they therefore appear as `sda` and `sdc`. They are partitioned identically along the following scheme:

```
# fdisk -l /dev/sda

Disk /dev/sda: 300 GB, 300090728448 bytes, 586114704 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
```

```
Disk identifier: 0x00039a9f

Device    Boot     Start       End   Sectors Size Id Type
/dev/sda1 *         2048   1992060   1990012 1.0G fd Linux raid autodetect
/dev/sda2        1992061   3984120   1992059 1.0G 82 Linux swap / Solaris
/dev/sda3        4000185 586099395 582099210 298G 5  Extended
/dev/sda5        4000185 203977305 199977120 102G fd Linux raid autodetect
/dev/sda6      203977306 403970490 199993184 102G fd Linux raid autodetect
/dev/sda7      403970491 586099395 182128904  93G 8e Linux LVM
```

- The first partitions of both disks (about 1 GB) are assembled into a RAID-1 volume, `md0`. This mirror is directly used to store the root filesystem.
- The `sda2` and `sdc2` partitions are used as swap partitions, providing a total 2 GB of swap space. With 1 GB of RAM, the workstation has a comfortable amount of available memory.
- The `sda5` and `sdc5` partitions, as well as `sda6` and `sdc6`, are assembled into two new RAID-1 volumes of about 100 GB each, `md1` and `md2`. Both these mirrors are initialized as physical volumes for LVM, and assigned to the `vg_raid` volume group. This VG thus contains about 200 GB of safe space.
- The remaining partitions, `sda7` and `sdc7`, are directly used as physical volumes, and assigned to another VG called `vg_bulk`, which therefore ends up with roughly 200 GB of space.

Once the VGs are created, they can be partitioned in a very flexible way. One must keep in mind that LVs created in `vg_raid` will be preserved even if one of the disks fails, which will not be the case for LVs created in `vg_bulk`; on the other hand, the latter will be allocated in parallel on both disks, which allows higher read or write speeds for large files.

We will therefore create the `lv_var` and `lv_home` LVs on `vg_raid`, to host the matching filesystems; another large LV, `lv_movies`, will be used to host the definitive versions of movies after editing. The other VG will be split into a large `lv_rushes`, for data straight out of the digital video cameras, and a `lv_tmp` for temporary files. The location of the work area is a less straightforward choice to make: while good performance is needed for that volume, is it worth risking losing work if a disk fails during an editing session? Depending on the answer to that question, the relevant LV will be created on one VG or the other.

We now have both some redundancy for important data and much flexibility in how the available space is split across the applications.

NOTE

Why three RAID-1 volumes?

We could have set up one RAID-1 volume only, to serve as a physical volume for `vg_raid`. Why create three of them, then?

The rationale for the first split (`md0` vs. the others) is about data safety: data written to both elements of a RAID-1 mirror are exactly the same, and it is therefore possible to bypass the RAID layer and mount one of the disks directly. In case of a kernel bug, for instance, or if the LVM metadata become corrupted, it is still possible to boot a minimal system to access critical data such as the layout of disks in

the RAID and LVM volumes; the metadata can then be reconstructed and the files can be accessed again, so that the system can be brought back to its nominal state.

The rationale for the second split (`md1` vs. `md2`) is less clear-cut, and more related to acknowledging that the future is uncertain. When the workstation is first assembled, the exact storage requirements are not necessarily known with perfect precision; they can also evolve over time. In our case, we can't know in advance the actual storage space requirements for video rushes and complete video clips. If one particular clip needs a very large amount of rushes, and the VG dedicated to redundant data is less than halfway full, we can re-use some of its unneeded space. We can remove one of the physical volumes, say `md2`, from `vg_raid` and either assign it to `vg_bulk` directly (if the expected duration of the operation is short enough that we can live with the temporary drop in performance), or undo the RAID setup on `md2` and integrate its components `sda6` and `sdc6` into the bulk VG (which grows by 200 GB instead of 100 GB); the `lv_rushes` logical volume can then be grown according to requirements.

12.2. Virtualization

Virtualization is one of the most major advances in the recent years of computing. The term covers various abstractions and techniques simulating virtual computers with a variable degree of independence on the actual hardware. One physical server can then host several systems working at the same time and in isolation. Applications are many, and often derive from this isolation: test environments with varying configurations for instance, or separation of hosted services across different virtual machines for security.

There are multiple virtualization solutions, each with its own pros and cons. This book will focus on Xen, LXC, and KVM, but other noteworthy implementations include the following:

- QEMU is a software emulator for a full computer; performances are far from the speed one could achieve running natively, but this allows running unmodified or experimental operating systems on the emulated hardware. It also allows emulating a different hardware architecture: for instance, an *amd64* system can emulate an *arm* computer. QEMU is free software.

  ➨ `https://www.qemu.org/`

- Bochs is another free virtual machine, but it only emulates the x86 architectures (i386 and amd64).

- VMWare is a proprietary virtual machine; being one of the oldest out there, it is also one of the most widely-known. It works on principles similar to QEMU. VMWare proposes advanced features such as snapshotting a running virtual machine.

  ➨ `https://www.vmware.com/`

- VirtualBox is a virtual machine that is mostly free software (some extra components are available under a proprietary license). Unfortunately it is in Debian's "contrib" section because it includes some precompiled files that cannot be rebuilt without a proprietary

compiler and it currently only resides in Debian Unstable as Oracle's policies make it impossible to keep it secure in a Debian stable release (see #794466[1]). While younger than VMWare and restricted to the i386 and amd64 architectures, it still includes some snapshotting and other interesting features.

➨ https://www.virtualbox.org/

HARDWARE

Virtualization support

Some computers might not have hardware virtualization support; when they do, it should be enabled in the BIOS.

To know if you have virtualization support enabled, you can check if the relevant flag is enabled with `grep`. If the following command for your processor returns some text, you already have virtualization support enabled:

- For Intel processors you can execute `grep vmx /proc/cpuinfo`
- For AMD processors you can execute `grep svm /proc/cpuinfo`

12.2.1. Xen

Xen is a "paravirtualization" solution. It introduces a thin abstraction layer, called a "hypervisor", between the hardware and the upper systems; this acts as a referee that controls access to hardware from the virtual machines. However, it only handles a few of the instructions, the rest is directly executed by the hardware on behalf of the systems. The main advantage is that performances are not degraded, and systems run close to native speed; the drawback is that the kernels of the operating systems one wishes to use on a Xen hypervisor need to be adapted to run on Xen.

Let's spend some time on terms. The hypervisor is the lowest layer, that runs directly on the hardware, even below the kernel. This hypervisor can split the rest of the software across several *domains*, which can be seen as so many virtual machines. One of these domains (the first one that gets started) is known as *dom0*, and has a special role, since only this domain can control the hypervisor and the execution of other domains. These other domains are known as *domU*. In other words, and from a user point of view, the *dom0* matches the "host" of other virtualization systems, while a *domU* can be seen as a "guest".

CULTURE

Xen and the various versions of Linux

Xen was initially developed as a set of patches that lived out of the official tree, and not integrated to the Linux kernel. At the same time, several upcoming virtualization systems (including KVM) required some generic virtualization-related functions to facilitate their integration, and the Linux kernel gained this set of functions (known as the *paravirt_ops* or *pv_ops* interface). Since the Xen patches were duplicating some of the functionality of this interface, they couldn't be accepted officially.

Xensource, the company behind Xen, therefore had to port Xen to this new framework, so that the Xen patches could be merged into the official Linux kernel. That meant a lot of code rewrite, and although Xensource soon had a working version

[1] https://bugs.debian.org/794466